TP Projet 3 : HR Agent et MDM Ressources

HESTIM | Data Governance | Projet "Enterprise Multi-Agent AI Governance Platform"
Membre : HR | Dataset : IBM HR Analytics Employee Attrition & Performance | Prérequis : TP0 terminé

Objectifs pédagogiques

À l'issue de ce TP, vous serez capables de :

- Inspecter un dataset et repérer les colonnes **codées** et les colonnes **inutiles**
- Charger un dataset brut dans PostgreSQL avec Python
- Documenter un dataset avec les 4 types de métadonnées (fiche catalogue)
- Rédiger un dictionnaire de données, en particulier les **valeurs admises** des codes
- Identifier les **données personnelles (PII)** et les classer dans OpenMetadata
- Appliquer les 5 tests du MDM et séparer les **attributs maîtres** des **faits** d'un employé
- Construire le Golden Record `employee` et ses référentiels (Department, JobRole...)
- Appliquer une règle de priorité quand une deuxième source modifie un employé

Votre mission

Vous êtes le **Data Steward du domaine RH**. Votre dataset est **le plus sensible du projet** : Confidential, sensibilité High, avec des données personnelles. Le HR Agent (partie 2) répondra à des questions comme "*Quel département a le plus fort taux d'attrition ?*", mais il ne devra **jamais** révéler le salaire d'une personne précise. Pour cela, il faut :

1. que les données RH soient chargées, **documentées** et que chaque colonne personnelle soit **marquée PII** ;
2. que chaque employé, département et poste existe **une seule fois** (Golden Records) ;
3. que les codes (1, 2, 3, 4) aient une **signification officielle** dans des référentiels.

Votre MDM s'appelle **Ressources** (domaine **Parties** pour les employés, **Reference Data** pour les codes) :

- **Employee** : l'employé. Données réelles.
- **Department**, **JobRole**, **EducationField** : départements, postes, filières. Données réelles.
- **Office** : les sites de l'entreprise. Petite table **simulée** (3 lignes).

Élément	Valeur
Dataset	`ibm_hr_analytics`
Classification	Confidential
Sensibilité	High
PII	Oui (âge, genre, situation familiale, salaire...)
Data Owner	Direction RH
Data Steward	Vous

Élément	Valeur
Relecteur de vos PR	Membre Finance

Concepts clés pour ce TP

> **DÉFINITION : PII (Personally Identifiable Information)**
> Une information qui permet d'identifier une personne, **seule ou combinée** avec d'autres. Ici, aucune colonne ne contient de nom. Mais "femme, 41 ans, divorcée, département RH" peut suffire à reconnaître quelqu'un dans une entreprise de 1 470 personnes. C'est pourquoi l'âge, le genre et la situation familiale sont des PII.

> **DÉFINITION : Attribut maître et fait**
> Dans une fiche employé, certains attributs sont **stables** et partagés (matricule, département, poste) : ce sont des **attributs maîtres**. D'autres changent souvent ou décrivent une situation à un instant (salaire du mois, satisfaction, départ ou non) : ce sont des **faits**. Le MDM garde les premiers, les seconds vont dans une table de données métier.

> **DÉFINITION : Valeurs admises**
> La liste exhaustive des valeurs autorisées pour un champ, chacune avec son sens. Votre dataset contient `Education = 3`. Sans dictionnaire, personne ne sait que 3 veut dire "Bachelor". C'est exactement l'exemple `STATUT_CLIENT = 3` du cours.

ÉTAPE 1 : Découvrir le dataset

1.1 Télécharger

1. Sur Kaggle, chercher **"IBM HR Analytics Employee Attrition & Performance"**.
2. Télécharger le fichier `WA_Fn-UseC_-HR-Employee-Attrition.csv`.
3. Le renommer `hr_attrition.csv` et le placer dans `data/raw/hr/`.

> **ATTENTION**
> Ce dataset est **fictif** (créé par des data scientists d'IBM), mais on le traite **comme s'il était réel** : c'est tout l'intérêt de l'exercice. Il ne va jamais sur GitHub (`data/raw/` est ignoré).

1.2 Inspecter les vraies colonnes

Créez `scripts/explore_hr.py` :

```
# Affiche les colonnes, les valeurs distinctes et les colonnes constantes
import pandas as pd

df = pd.read_csv("data/raw/hr/hr_attrition.csv")
print(df.shape)                    # (lignes, colonnes)
print(df.dtypes)
```

```
print("\nValeurs manquantes :", df.isna().sum().sum())

# Colonnes texte : quelles valeurs ?
for col in df.select_dtypes("object").columns:
    print(f"\n{col} :", df[col].unique())

# Colonnes qui ont toujours la meme valeur
print("\nColonnes constantes :", [c for c in df.columns if df[c].nunique() == 1])
```

```
python scripts/explore_hr.py
```

Questions à noter dans votre compte rendu :

1. Combien de lignes et de colonnes ? Y a-t-il des valeurs manquantes ?
2. Quelle colonne identifie un employé de façon unique ?
3. Quelles colonnes sont **constantes** ? Servent-elles à quelque chose ?
4. Quels sont les 3 départements ? Combien de postes (`JobRole`) différents ?
5. Le poste `Manager` apparaît-il dans un seul département ? (indice : `df.groupby("JobRole")["Department"].unique()`)
6. Quelles colonnes sont des **codes chiffrés** (1 à 4 ou 1 à 5) dont le sens n'est pas écrit dans le fichier ?

> **CAPTURE D'ÉCRAN REQUISE**
> La sortie du script.

ÉTAPE 2 : Charger les données brutes dans PostgreSQL

On charge une **copie brute** dans `data.hr_employee_raw` . Une seule modification : les noms de colonnes passent en `snake_case` (`MonthlyIncome` → `monthly_income`), selon notre standard commun.

Installez les dépendances Python (`sqlalchemy` est déjà dans `requirements.txt`) :

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Créez `scripts/load_hr.py` :

```
# Charge le dataset IBM HR brut dans PostgreSQL (data.hr_employee_raw)
import os
import re
import pandas as pd
from dotenv import load_dotenv
from sqlalchemy import create_engine

load_dotenv()
engine = create_engine(
    f"postgresql+psycopg2://{os.getenv('POSTGRES_USER')}:{os.getenv('POSTGRES_PASSWORD')}"
    f"@localhost:{os.getenv('POSTGRES_PORT')}/{os.getenv('POSTGRES_DB')}"
)

df = pd.read_csv("data/raw/hr/hr_attrition.csv", encoding="utf-8-sig")
# MonthlyIncome -> monthly_income (standard commun de nommage)
```

```
df.columns = [re.sub(r"(?<!^)(?=[A-Z])", "_", c).lower() for c in df.columns]

df.to_sql("hr_employee_raw", engine, schema="data", if_exists="replace", index=False)
print("hr_employee_raw :", len(df), "lignes,", len(df.columns), "colonnes")
print(list(df.columns))
```

```
python scripts/load_hr.py
docker exec gouv_postgres psql -U gouv -d governance -c "SELECT COUNT(*) FROM data.hr_employee_raw;"
```

Vous devez obtenir **1 470** lignes.

> **EXPLICATION : pourquoi ne pas corriger les données dès maintenant ?**
> La table brute est la **preuve** de ce qu'on a reçu. Si plus tard un chiffre est contesté, on peut toujours revenir à la source. Les corrections et la séparation maître / faits se font dans le MDM (étape 5), dans des tables séparées.

> **CAPTURE D'ÉCRAN REQUISE**
> La sortie du script et le `COUNT(*)`.

Commit : `TP3 etape 2 : chargement brut du dataset IBM HR`

ÉTAPE 3 : Documenter le dataset (métadonnées et dictionnaire)

3.1 La fiche catalogue (niveau dataset)

Créez `docs/dictionnaire/ibm_hr_analytics.md` et commencez par cette fiche. Complétez les `...`.

Type	Attribut	Valeur
Métier	Nom métier	Dossiers employés : attrition et performance
Métier	Description	... (2 phrases)
Métier	Domaine	RH
Technique	Table	`data.hr_employee_raw`
Technique	Format source	CSV, séparateur virgule, 35 colonnes
Opérationnelle	Source	Kaggle, dataset fictif IBM
Opérationnelle	Fréquence de mise à jour	Aucune (photo unique)
Opérationnelle	Volumétrie	1 470 lignes
Gouvernance	Data Owner	Direction RH
Gouvernance	Data Steward	... (votre nom)
Gouvernance	Classification	Confidential
Gouvernance	Sensibilité	High
Gouvernance	PII	Oui : `age`, `gender`, `marital_status`, `monthly_income`, `daily_rate`, `hourly_rate`, `monthly_rate`, `employee_number`

3.2 Le dictionnaire de données (niveau colonne)

Documentez **au moins 10 colonnes**, dont **obligatoirement** toutes les colonnes codées. Voici les valeurs officielles du dataset :

Colonne	Valeurs admises
`education`	1 = Below College, 2 = College, 3 = Bachelor, 4 = Master, 5 = Doctor
`environment_satisfaction`, `job_satisfaction`, `relationship_satisfaction`, `job_involvement`	1 = Low, 2 = Medium, 3 = High, 4 = Very High
`performance_rating`	1 = Low, 2 = Good, 3 = Excellent, 4 = Outstanding
`work_life_balance`	1 = Bad, 2 = Good, 3 = Better, 4 = Best

Exemple rempli :

Nom physique	Nom métier	Définition	Type	Contraintes	Valeurs admises	Règle de gestion	PII
`monthly_income`	Salaire mensuel	Rémunération brute mensuelle de l'employé	Entier (USD)	NOT NULL	Nombre strictement positif	Ne doit jamais être communiqué individuellement hors Direction RH	**Oui**
`attrition`	Départ de l'employé	Indique si l'employé a quitté l'entreprise	Texte	NOT NULL	`Yes` = a quitté, `No` = toujours présent	...	Non

> **EXPLICATION : les 4 règles de rédaction du cours**
>
> 1. **La définition** est une phrase complète, sans jargon technique.
> 2. **La règle de gestion** commence par un verbe et se vérifie.
> 3. **Les valeurs admises** sont exhaustives : chaque code a un sens écrit.
> 4. **Le propriétaire** est toujours une direction métier (Direction RH), jamais l'IT.

Ajoutez une section **"Colonnes constantes"** : `employee_count`, `over18`, `standard_hours` ont toujours la même valeur. Notez votre décision (on ne les reprend pas dans le MDM) et pourquoi.

Commit : `TP3 etape 3 : fiche catalogue et dictionnaire IBM HR`

ÉTAPE 4 : Cataloguer dans OpenMetadata

4.1 Ingestion

1. **Paramètres, Services, Bases de données**, ouvrir `gouv_postgres` (créé au TP0).
2. Onglet **Ingestions** (ou **Agents** selon la version), **Ajouter une ingestion de métadonnées**.
3. Filtre de schémas : inclure `data`, `master`, `ref`.
4. Lancer l'ingestion et attendre le statut **Success**.

5. Dans **Explorer**, chercher `hr_employee_raw`.

4.2 Enrichir la table brute

Champ OpenMetadata	Valeur
Description	Reprendre votre fiche catalogue
Owner	Équipe `Direction RH`
Domaine	`RH`
Tags	`Confidentialite.Confidential`, `Sensibilite.High`

4.3 Marquer les colonnes PII (le plus important de ce TP)

Sur **chacune** des 8 colonnes PII de votre fiche, ajoutez le tag `PII.Sensitive` et sa description. Sur quelques colonnes non personnelles (`department`, `job_role`, `attrition`), ajoutez `PII.NonSensitive`.

> **EXPLICATION : à quoi sert ce marquage ?**
> Dans la partie 2, la Governance Engine lira ces tags. Règle prévue : le HR Agent peut répondre "taux d'attrition par département" (un **agrégat**, aucune personne visible), mais une demande sur `monthly_income` individuel sera **refusée**. Le tag PII n'est donc pas décoratif : c'est lui qui déclenchera le refus.

4.4 Glossaire

Dans `Glossaire Projet`, créez :

Terme	Définition	Lier à
Employee	Personne liée à l'entreprise par un contrat de travail	table `master.employee` (étape 5)
Attrition	Départ d'un employé de l'entreprise, volontaire ou non	colonne `attrition`
Department	Unité organisationnelle de rattachement d'un employé	table `master.department`
Performance Rating	Évaluation annuelle de la performance, de 1 (Low) à 4 (Outstanding)	colonne `performance_rating`

> **CAPTURE D'ÉCRAN REQUISE**
> La page de `hr_employee_raw` avec les tags, et la liste des colonnes avec les tags `PII.Sensitive` visibles. Le glossaire avec vos 4 termes.

ÉTAPE 5 : Construire le MDM Ressources

5.1 Choisir les entités maîtres avec les 5 tests

Complétez dans votre compte rendu (OUI / NON + justification courte) :

Entité / attribut	Nature	Volatilité faible	Réutilisation	Partage	Criticité	Master Data ?
Employee (matricule, genre, poste)	...	...	...	...	...	...
Department	...	...	...	...	...	...
JobRole	...	...	...	...	...	...
MonthlyIncome	...	...	...	...	...	...
Attrition	...	...	...	...	...	...

> **EXPLICATION : le résultat attendu**
> Employee, Department et JobRole sont des **entités** stables, partagées (RH, paie, et dans la partie 2 la gestion des utilisateurs de la plateforme). Le salaire change à chaque augmentation, et l'attrition est un **événement** (un départ) : ce sont des **faits**. Ils iront dans `data.hr_employee_facts`, qui pointe vers `master.employee`. L'âge aussi change chaque année : c'est un fait, pas un attribut maître.

5.2 Le modèle

```
  master.office (simulé)
         |
  master.department    master.job_role    master.education_field
          \                  |                  /
           +--------- master.employee ---------+---- ref.education_level
                     (Golden Record)
                           |
                 data.hr_employee_facts  ---- ref.satisfaction_level
          (salaire, satisfaction, attrition)   ref.performance_rating
                                               ref.work_life_balance
```

5.3 Les tables simulées

Créez `data/seed/office.csv` (fictif, peut aller dans Git) :

```
code,city,country
OFF-CAS,Casablanca,MA
OFF-RAB,Rabat,MA
OFF-TNG,Tanger,MA
```

Créez aussi `data/seed/hr_mobility.csv`. C'est une **deuxième source simulée** : le logiciel de mobilité interne, qui annonce que 3 employés ont changé de poste.

```
employee_number,department,job_role,effective_date
1,Research & Development,Research Scientist,2026-09-01
2,Sales,Sales Executive,2026-10-01
4,Human Resources,Human Resources,2026-08-15
```

5.4 Le script MDM

Créez `sql/31_hr_mdm.sql` :

```
-- MDM Ressources (membre HR)
-- Pre-requis : data.hr_employee_raw charge
```

```
-- 1. Reference Data : la signification officielle des codes ---------------
CREATE TABLE IF NOT EXISTS ref.education_level (
  level SMALLINT PRIMARY KEY, label VARCHAR(30) NOT NULL);
INSERT INTO ref.education_level VALUES
  (1,'Below College'), (2,'College'), (3,'Bachelor'), (4,'Master'), (5,'Doctor')
ON CONFLICT DO NOTHING;

CREATE TABLE IF NOT EXISTS ref.satisfaction_level (
  level SMALLINT PRIMARY KEY, label VARCHAR(30) NOT NULL);
INSERT INTO ref.satisfaction_level VALUES
  (1,'Low'), (2,'Medium'), (3,'High'), (4,'Very High')
ON CONFLICT DO NOTHING;

CREATE TABLE IF NOT EXISTS ref.performance_rating (
  level SMALLINT PRIMARY KEY, label VARCHAR(30) NOT NULL);
INSERT INTO ref.performance_rating VALUES
  (1,'Low'), (2,'Good'), (3,'Excellent'), (4,'Outstanding')
ON CONFLICT DO NOTHING;

CREATE TABLE IF NOT EXISTS ref.work_life_balance (
  level SMALLINT PRIMARY KEY, label VARCHAR(30) NOT NULL);
INSERT INTO ref.work_life_balance VALUES
  (1,'Bad'), (2,'Good'), (3,'Better'), (4,'Best')
ON CONFLICT DO NOTHING;

-- 2. Referentiels de l'organisation -----------------------------------
CREATE TABLE IF NOT EXISTS master.office (
  office_id SERIAL PRIMARY KEY,
  code      VARCHAR(10) UNIQUE NOT NULL,
  city      VARCHAR(60) NOT NULL,
  country   CHAR(2));
TRUNCATE master.office RESTART IDENTITY CASCADE;
COPY master.office (code, city, country)
  FROM '/seed/office.csv' WITH (FORMAT csv, HEADER true);

CREATE TABLE IF NOT EXISTS master.department (
  department_id SERIAL PRIMARY KEY,
  label         VARCHAR(60) UNIQUE NOT NULL,
  office_id     INT REFERENCES master.office(office_id));
INSERT INTO master.department (label)
SELECT DISTINCT TRIM(department) FROM data.hr_employee_raw
ON CONFLICT DO NOTHING;
-- Rattachement simule des departements aux sites
UPDATE master.department d SET office_id = o.office_id
FROM master.office o
WHERE (d.label = 'Sales' AND o.code = 'OFF-CAS')
   OR (d.label = 'Research & Development' AND o.code = 'OFF-RAB')
   OR (d.label = 'Human Resources' AND o.code = 'OFF-CAS');

CREATE TABLE IF NOT EXISTS master.job_role (
  job_role_id SERIAL PRIMARY KEY,
  label       VARCHAR(60) UNIQUE NOT NULL);
INSERT INTO master.job_role (label)
SELECT DISTINCT TRIM(job_role) FROM data.hr_employee_raw
ON CONFLICT DO NOTHING;

CREATE TABLE IF NOT EXISTS master.education_field (
  education_field_id SERIAL PRIMARY KEY,
  label              VARCHAR(60) UNIQUE NOT NULL);
INSERT INTO master.education_field (label)
```

```
SELECT DISTINCT TRIM(education_field) FROM data.hr_employee_raw
ON CONFLICT DO NOTHING;

-- 3. Golden Record Employee (attributs maitres seulement) -----------------
CREATE TABLE IF NOT EXISTS master.employee (
  employee_id        SERIAL PRIMARY KEY,
  employee_number    INT UNIQUE NOT NULL,
  gender             VARCHAR(10),
  marital_status     VARCHAR(10),
  department_id      INT REFERENCES master.department(department_id),
  job_role_id        INT REFERENCES master.job_role(job_role_id),
  job_level          SMALLINT,
  education_level    SMALLINT REFERENCES ref.education_level(level),
  education_field_id INT REFERENCES master.education_field(education_field_id),
  source             VARCHAR(20) NOT NULL DEFAULT 'IBM_HR',
  updated_at         TIMESTAMP DEFAULT now());

INSERT INTO master.employee (employee_number, gender, marital_status, department_id,
                             job_role_id, job_level, education_level, education_field_id)
SELECT r.employee_number, TRIM(r.gender), TRIM(r.marital_status),
       d.department_id, j.job_role_id, r.job_level, r.education, f.education_field_id
FROM data.hr_employee_raw r
JOIN master.department d      ON d.label = TRIM(r.department)
JOIN master.job_role j        ON j.label = TRIM(r.job_role)
JOIN master.education_field f ON f.label = TRIM(r.education_field)
ON CONFLICT (employee_number) DO NOTHING;

-- 4. Deuxieme source : la mobilite interne --------------------------------
-- Regle de priorite : pour le departement et le poste, la source MOBILITE
-- (plus recente) gagne sur le fichier RH d'origine.
CREATE TEMP TABLE mobility (employee_number INT, department TEXT,
                            job_role TEXT, effective_date DATE);
COPY mobility FROM '/seed/hr_mobility.csv' WITH (FORMAT csv, HEADER true);

UPDATE master.employee e
SET department_id = d.department_id,
    job_role_id   = j.job_role_id,
    source        = 'MOBILITY',
    updated_at    = now()
FROM mobility m
JOIN master.department d ON d.label = m.department
JOIN master.job_role j   ON j.label = m.job_role
WHERE e.employee_number = m.employee_number;

-- 5. Faits RH (donnees metier qui pointent vers le MDM) --------------------
CREATE TABLE IF NOT EXISTS data.hr_employee_facts (
  employee_id               INT PRIMARY KEY REFERENCES master.employee(employee_id),
  age                       SMALLINT,
  monthly_income            INT,
  daily_rate                INT,
  hourly_rate               INT,
  monthly_rate              INT,
  attrition                 BOOLEAN,
  over_time                 BOOLEAN,
  business_travel           VARCHAR(30),
  distance_from_home        SMALLINT,
  environment_satisfaction  SMALLINT REFERENCES ref.satisfaction_level(level),
  job_satisfaction          SMALLINT REFERENCES ref.satisfaction_level(level),
  relationship_satisfaction SMALLINT REFERENCES ref.satisfaction_level(level),
  job_involvement           SMALLINT REFERENCES ref.satisfaction_level(level),
  work_life_balance         SMALLINT REFERENCES ref.work_life_balance(level),
```

```
  performance_rating        SMALLINT REFERENCES ref.performance_rating(level),
  percent_salary_hike       SMALLINT,
  stock_option_level        SMALLINT,
  num_companies_worked      SMALLINT,
  total_working_years       SMALLINT,
  training_times_last_year  SMALLINT,
  years_at_company          SMALLINT,
  years_in_current_role     SMALLINT,
  years_since_last_promotion SMALLINT,
  years_with_curr_manager   SMALLINT);

TRUNCATE data.hr_employee_facts;
INSERT INTO data.hr_employee_facts
SELECT e.employee_id, r.age, r.monthly_income, r.daily_rate, r.hourly_rate,
       r.monthly_rate, r.attrition = 'Yes', r.over_time = 'Yes', r.business_travel,
       r.distance_from_home, r.environment_satisfaction, r.job_satisfaction,
       r.relationship_satisfaction, r.job_involvement, r.work_life_balance,
       r.performance_rating, r.percent_salary_hike, r.stock_option_level,
       r.num_companies_worked, r.total_working_years, r.training_times_last_year,
       r.years_at_company, r.years_in_current_role, r.years_since_last_promotion,
       r.years_with_curr_manager
FROM data.hr_employee_raw r
JOIN master.employee e ON e.employee_number = r.employee_number;
-- employee_count, over18, standard_hours : constantes, non reprises (voir dictionnaire)
```

Exécutez-le :

```
docker exec gouv_postgres psql -U gouv -d governance -v ON_ERROR_STOP=1 -f /sql/31_hr_mdm.sql
```

À la deuxième exécution, des messages `NOTICE: ... already exists, skipping` apparaissent : c'est normal, le script est prévu pour être relancé sans danger.

> **EXPLICATION : les étapes du Golden Record, dans le script**
>
> 1. **Charger** deux sources : le fichier RH d'origine et la mobilité interne (simulée).
> 2. **Standardiser** : `TRIM()` sur les libellés, et des **référentiels** pour donner un sens officiel aux codes.
> 3. **Dédoublonner** : `employee_number UNIQUE`, et un département ou un poste n'existe qu'une fois (`label UNIQUE`).
> 4. **Appliquer la règle de priorité** : la mobilité, plus récente, écrase le département et le poste. La colonne `source` garde la trace de qui a gagné.
> 5. **Publier** `master.employee` : les faits pointent vers lui par `employee_id`.

5.5 Contrôles qualité du référentiel

Créez `sql/32_hr_quality.sql` et exécutez-le de la même façon :

```
-- Q1. Unicite : un matricule n'apparait jamais deux fois (attendu : 0 ligne)
SELECT employee_number, COUNT(*) FROM master.employee
GROUP BY employee_number HAVING COUNT(*) > 1;

-- Q2. Exhaustivite : tous les employes de la source sont dans le Golden Record
SELECT (SELECT COUNT(*) FROM data.hr_employee_raw) AS nb_source,
       (SELECT COUNT(*) FROM master.employee)      AS nb_master,
       (SELECT COUNT(*) FROM data.hr_employee_facts) AS nb_faits;
```

```
-- Q3. Completude des attributs maitres (seuil : 95 %)
SELECT ROUND(100.0 * COUNT(department_id)   / COUNT(*), 2) AS pct_department,
       ROUND(100.0 * COUNT(job_role_id)     / COUNT(*), 2) AS pct_job_role,
       ROUND(100.0 * COUNT(education_level) / COUNT(*), 2) AS pct_education
FROM master.employee;

-- Q4. La regle de priorite a-t-elle ete appliquee ?
SELECT e.employee_number, d.label AS department, j.label AS job_role, e.source
FROM master.employee e
JOIN master.department d ON d.department_id = e.department_id
JOIN master.job_role j   ON j.job_role_id = e.job_role_id
WHERE e.employee_number IN (1, 2, 4);

-- Q5. Un poste dans plusieurs departements ?
SELECT j.label, ARRAY_AGG(DISTINCT d.label) AS departements
FROM master.employee e
JOIN master.job_role j   ON j.job_role_id = e.job_role_id
JOIN master.department d ON d.department_id = e.department_id
GROUP BY j.label HAVING COUNT(DISTINCT d.label) > 1;

-- Q6. Apercu de ce que le HR Agent pourra donner (un AGREGAT, sans personne visible)
SELECT d.label AS department,
       COUNT(*) AS nb_employes,
       ROUND(100.0 * AVG(f.attrition::INT), 1) AS taux_attrition_pct
FROM data.hr_employee_facts f
JOIN master.employee e   ON e.employee_id = f.employee_id
JOIN master.department d ON d.department_id = e.department_id
GROUP BY d.label ORDER BY taux_attrition_pct DESC;
```

Ce que vous devez faire avec les résultats (c'est le travail du Steward) :

- **Q2** : les trois nombres doivent être égaux (1 470). Sinon, trouvez quelles lignes manquent et pourquoi.
- **Q4** : vérifiez que les employés 1, 2 et 4 ont bien `source = 'MOBILITY'` et leur nouveau poste.
- **Q5** : vous devriez voir que `Manager` existe dans les 3 départements. Expliquez pourquoi on a fait de `job_role` un référentiel **indépendant** du département.
- **Q6** : notez quel département a le plus fort taux d'attrition. C'est la réponse que donnera le HR Agent, **sans jamais** montrer une personne.

> **CAPTURE D'ÉCRAN REQUISE**
> Les résultats de Q2, Q4, Q5 et Q6.

5.6 Mettre à jour le catalogue

1. Relancez l'ingestion OpenMetadata : les tables `master.*`, `ref.*` et `data.hr_employee_facts` apparaissent.
2. Classez chaque nouvelle table. **Attention, la classification n'est pas la même partout :**

Table	Classification	Sensibilité	Pourquoi
`master.employee`	Confidential	High	contient genre et situation familiale (PII)
`data.hr_employee_facts`	Confidential	High	contient âge et salaires (PII)
`master.department`, `master.job_role`, `master.education_field`, `master.office`	Internal	Low	aucune personne, juste l'organisation

Table	Classification	Sensibilité	Pourquoi
`ref.education_level`, `ref.satisfaction_level` …	Internal	Low	des codes et leur sens

3. Ajoutez `PII.Sensitive` sur `gender`, `marital_status` (dans `master.employee`) et sur `age`, `monthly_income`, `daily_rate`, `hourly_rate`, `monthly_rate` (dans `data.hr_employee_facts`).
4. Owner `Direction RH`, domaine `RH` sur toutes ces tables. Précisez **"données simulées"** dans la description de `master.office`.
5. Liez les termes **Employee** et **Department** à leurs tables.
6. (Optionnel) Onglet **Lignage** : `data.hr_employee_raw` → `master.employee` → `data.hr_employee_facts`.

Commit : `TP3 etape 5 : MDM Ressources et controles qualite` Puis ouvrez une **Pull Request** vers `main`, relue par le membre Finance.

Questions de compréhension (à préparer pour l'oral)

1. Aucune colonne ne contient de nom. Pourquoi ce dataset contient-il quand même des PII ?
2. Pourquoi le salaire et l'attrition ne sont-ils pas dans `master.employee` ?
3. Quelle est votre règle de priorité ? Qu'indique la colonne `source` ?
4. Pourquoi `master.department` est Internal alors que `master.employee` est Confidential ?
5. Que se passerait-il si le code `education = 3` n'était documenté nulle part ?
6. Pourquoi la requête Q6 pourra-t-elle être autorisée au HR Agent, mais pas `SELECT monthly_income WHERE employee_number = 1` ?

Livrables

- [ ] `scripts/explore_hr.py`, `scripts/load_hr.py`
- [ ] `docs/dictionnaire/ibm_hr_analytics.md` (fiche + 10 colonnes minimum + colonnes constantes)
- [ ] `data/seed/office.csv`, `data/seed/hr_mobility.csv`
- [ ] `sql/31_hr_mdm.sql`, `sql/32_hr_quality.sql`
- [ ] Tableau des 5 tests rempli
- [ ] Captures d'écran des étapes 1, 2, 4 et 5
- [ ] Pull Request relue par le membre Finance

Suite (partie 2) : tables de gouvernance (vous porterez le référentiel d'identité : utilisateurs, rôles, agents), Governance Engine, HR Agent (règles A3 champ sensible et A4 agrégat).